

DEV BHOOMI UTTARAKHAND UNIVERSITY

Dehradun, Uttarakhand

(Notified by Govt. of India u/s 2(f) of the U.G.C. Act, 1956)

Academic Year: 2025–2026

OOPs Using C++

2nd Semester | BCA (AI&DS)

ON THE TOPIC OF

Plagiarism Checker

A C++ OOP-Based Console Plagiarism Detection System

Submitted by	Gaurav Gunjan
Course	BCA (AI & DS)
Semester	2nd Semester
Subject	OOPs Using C++
Faculty Name	Ms. Dhruvi Dagar
College	Dev Bhoomi Uttarakhand University
Submission Date	28 April 2026

Acknowledgement

First and foremost, I would like to express my sincere gratitude to the Almighty for providing me with the strength, knowledge, and determination to successfully complete this project.

I am deeply grateful to my project guide and faculty of the Department of Computer Applications, Dev Bhoomi Uttarakhand University, Dehradun, for their invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project. Their expertise and patience helped me navigate every challenge I encountered during this work.

I would also like to extend my heartfelt thanks to the Head of the Department and all faculty members of the Department of Computer Applications for their academic support and for creating a conducive learning environment. Their teachings formed the foundation upon which this project is built.

My special thanks go to my classmates and friends who provided motivation, assistance, and healthy discussions during the development phase of this project. Their feedback during testing proved extremely helpful in refining the application.

Last but not least, I am deeply indebted to my parents and family members for their unwavering moral support, encouragement, and belief in me throughout my academic journey. Without their blessings and support, this project would not have been possible.

Gaurav Gunjan

BCA (AI&DS) – 2nd Semester
Dev Bhoomi Uttarakhand University
April 2026

Table of Contents

Acknowledgement.....	
Table of Contents.....	
1. Introduction to Topic.....	
1.1 What is Plagiarism?	
1.2 Importance of Plagiarism Detection	
1.3 Overview of the Project	
2. Project Specification.....	
2.1 Objectives	
2.2 Scope	
3. Hardware Requirements	
4. Software Requirements.....	
5. OOP Design & Class Structure	
5.1 OOP Principles Applied	
5.2 Class Overview	
5.3 Document Class.....	
5.4 TerminalRenderer Class	
6. Console Interface & User Experience.....	
6.1 Console Interaction Model	
6.2 Console Output Structure	
7. Important Features of the Project	
8. Implementation.....	
8.1 Step-by-Step Working	
8.2 Core Algorithm Pseudocode	
9. System Architecture	
10. Coding and Libraries Used.....	
11. Sample Console Output	
12. Advantages and Limitations	
13. Future Enhancements.....	
14. Conclusion	
15. References.....	

Chapter 1: Introduction to Topic

1.1 What is Plagiarism?

In academia and professional writing, plagiarism refers to the act of presenting someone else's work, ideas, or words as one's own without proper attribution. It is considered a serious ethical and intellectual violation that undermines the integrity of academic institutions and professional communities. With the rapid growth of digital content and easy online access to information, plagiarism has become an increasingly significant concern for educators, researchers, and content creators worldwide.

Plagiarism can occur in many forms: verbatim copying of text, paraphrasing without citation, submitting another person's work, or self-plagiarism (reusing one's own previously submitted work). Regardless of the form, it erodes trust and intellectual honesty in educational and professional environments.

1.2 Importance of Plagiarism Detection

Automated plagiarism detection systems play a critical role in maintaining academic integrity. These tools compare submitted documents against reference files or other submitted works to identify similarities. Key reasons why plagiarism detection is important include:

- Protection of original work and intellectual property rights.
- Ensuring fair evaluation of students' efforts in academic settings.
- Maintaining the credibility and trustworthiness of academic institutions.
- Encouraging original thinking, research, and creative work.
- Deterring students and professionals from taking shortcuts in their work.

Similarity metrics such as Jaccard Similarity provide a mathematically sound basis for comparing two documents, offering a precise percentage reflecting the degree of overlap between their word sets.

1.3 Overview of the Project

Plagiarism Checker is a console-based plagiarism detection system developed as a BCA 2nd Semester OOPs project, implemented in C++ using Object-Oriented Programming principles. The system accepts two plain-text file paths as command-line arguments and prints a detailed similarity analysis to the terminal within seconds.

The project uses two primary classes: the Document class, which handles file reading, tokenization, normalization, and unique word set construction using the C++ Standard Library (vector and unordered_set); and the TerminalRenderer class, which handles all formatted console output. The Jaccard Similarity coefficient is used as the comparison metric.

The program is compiled using g++ via MinGW on Windows (or GCC on Linux/macOS) and executed directly in the terminal. There is no web server, Docker, or browser dependency — the entire interaction happens through the console, making it a true OOP compiler-based project.

Key Highlights:

- Accepts two .txt file paths as command-line arguments.
- OOP design with two well-defined classes: Document and TerminalRenderer.
- Stopword filtering removes common words (a, an, the) before comparison.
- Jaccard Similarity provides an industry-standard percentage similarity score.
- Console output includes score, verdict, matched words, and statistics.
- Compiled with g++ using the provided compile.bat (Windows) script.
- No external libraries — uses only the C++ Standard Library.

Chapter 2: Project Specification

2.1 Objectives

The primary objective of this project is to design and implement a C++ OOP-based plagiarism detection system that compares two text documents and computes their similarity. The specific objectives are:

1. To implement a C++ console application that accepts two .txt file paths via command-line arguments and performs word-level similarity analysis.
2. To apply OOP principles: encapsulation and abstraction through well-defined C++ classes (Document and TerminalRenderer).
3. To use the C++ Standard Library containers (vector, unordered_set) for efficient word storage and set operations.
4. To implement stopwords filtering that excludes common words (a, an, the) from the similarity computation.
5. To compute the Jaccard Similarity coefficient as the similarity metric between the two documents.
6. To present results clearly in the console: similarity score, verdict, word statistics, and matched word list.
7. To compile and run the project using a g++ compiler with the provided compile.bat script.

2.2 Scope

The scope of this project covers the design, implementation, and compilation of a console-based C++ plagiarism detection application intended for educational use and academic demonstrations.

Included in Scope:

- Input of two plain-text (.txt) file paths through command-line arguments.
- Word tokenization, normalization (lowercase, punctuation stripping), stopwords removal, and unique-set computation.
- Jaccard Similarity-based plagiarism detection with verdict labels (High, Moderate, Low, Minimal).
- Console display of matched words, statistics, and similarity score.
- Cross-platform compilation via g++ (Windows with MinGW, Linux, macOS).

Excluded from Scope:

- Support for file formats other than plain text (.txt).
- Graphical User Interface (GUI), web interface, or browser-based interaction.
- Database storage of comparison history or user profiles.
- Online plagiarism checking against internet content or external databases.

Chapter 3: Hardware Requirements

Plagiarism Checker is built in C++, a highly portable compiled language. The application compiles to a native binary (plagiarism.exe on Windows) and runs entirely in the terminal, making it extremely lightweight. It can run on any hardware that supports a C++ compiler.

Component	Minimum	Recommended
Processor	Any 1 GHz+ CPU	Intel Core i3 or higher
RAM	512 MB	2 GB or more
Hard Disk	50 MB free space	500 MB or more
Display	Any terminal display	1366 x 768 or higher
Input	Standard keyboard	Standard keyboard & mouse
Network	Not required	Not required
OS	Windows 10 / Linux / macOS	Ubuntu 20.04+ or Windows 11

Since the C++ backend compiles to a native binary with no external runtime dependencies, the application has a minimal resource footprint and runs efficiently on standard college lab computers and personal laptops.

Chapter 4: Software Requirements

The development and execution of the Plagiarism Checker require the following software components:

4.1 g++ Compiler (MinGW / GCC)

The GNU C++ compiler (g++) is required to compile the C++ source file (plagiarism.cpp) into a native executable (plagiarism.exe). g++ version 9.0 or higher is recommended. The compile.bat script (Windows) automates this process:

```
@echo off
title Plagiarism Checker CLI - Compiler
echo Compiling plagiarism.cpp...
g++ -O2 -o plagiarism.exe plagiarism.cpp
if %errorlevel% neq 0 goto fail
echo [SUCCESS] Created: plagiarism.exe
echo Usage: plagiarism.exe file1.txt file2.txt
```

4.2 MinGW (Windows Only)

On Windows, MinGW (Minimalist GNU for Windows) provides the GCC toolchain including g++. It must be installed and added to the system PATH so that compile.bat can locate g++ automatically. On Linux/macOS, GCC is typically pre-installed or available via the package manager.

4.3 Terminal / Command Prompt

The application runs entirely in the terminal (Command Prompt or PowerShell on Windows, Bash/Zsh on Linux/macOS). The user interacts by providing text file paths as command-line arguments:

```
plagiarism.exe document1.txt document2.txt
```

4.4 Text Editor / IDE (Optional)

Any text editor or IDE may be used to view or edit the source code — Visual Studio Code with the C/C++ extension, Code::Blocks, or any plain text editor. An IDE is not required; the project compiles and runs entirely from the command line.

4.5 Operating System

The application is cross-platform. Supported operating systems:

- Windows 10 / 11 (with MinGW for g++ support)
- Ubuntu Linux 20.04 / 22.04 (GCC installed by default)
- macOS 12 (Monterey) or later (with Xcode Command Line Tools)

Chapter 5: OOP Design & Class Structure

5.1 OOP Principles Applied

The Plagiarism Checker is fully implemented using Object-Oriented Programming in C++. All logic is encapsulated within two classes, each with a single clear responsibility — following the Single Responsibility Principle of OOP design.

- Encapsulation – Each class hides its internal data (private members) and exposes only the necessary public interface.
- Abstraction – The main() function only sees two classes; all internal processing is hidden behind clean method calls.
- Modularity – Document handles data; TerminalRenderer handles output. Neither class is responsible for both.

5.2 Class Overview

Class	Responsibility	Key Members / Methods
Document	Reads file, tokenizes, normalizes, builds word sets	getWords(), getUniqueWords(), getWordCount(), isEmpty()
TerminalRenderer	Formats and prints all console output	printHeader(), printStats(), printResult(), printMatches(), printError()

5.3 Document Class

The Document class encapsulates everything related to a single text file. It reads the file on construction, normalizes all words to lowercase, strips punctuation, filters out stopwords (a, an, the), and stores results in a `vector<string>` for ordered word access and an `unordered_set<string>` for $O(1)$ unique word lookups. All data members are private; external code accesses results only through getter methods.

Private members: `filename`, `content`, `words` (vector), `unique_words` (unordered_set), `stop_words` (unordered_set). The private `normalizeAndTokenize()` method is called from the constructor automatically, so the object is fully ready to use as soon as it is created.

5.4 TerminalRenderer Class

The TerminalRenderer class contains only static methods, meaning no object needs to be instantiated — methods are called directly on the class (e.g., `TerminalRenderer::printHeader()`). This design choice is appropriate since the renderer holds no state; it only formats and prints data passed to it as arguments. All output formatting (column alignment using `setw`, decimal precision using `setprecision`, verdict logic) is centralized here, keeping `main()` clean and readable.

Chapter 6: Console Interface & User Experience

6.1 Console Interaction Model

As a fully console-based application, all user interaction occurs through the terminal. There is no graphical window, web page, or browser required. The user compiles the project once using compile.bat and then runs the executable directly from the command prompt with two text file paths as arguments.

```
plagiarism.exe <file1.txt> <file2.txt>
```

If fewer than two arguments are provided, the program prints a usage error and exits gracefully with a non-zero exit code. If either file cannot be read, the Document object will have an empty word list, and the program will report an error via `TerminalRenderer::printError()`.

6.2 Console Output Structure

The program prints a structured, readable report to the console organized in the following sequence:

8. Header Banner – Prints the program name (Plagiarism Checker - Terminal Plagiarism Checker) inside a separator line.
9. Common Words – Lists up to 20 matched words found in both documents, with a count of additional matches if more than 20 exist.
10. Analysis Statistics – A table showing total words, unique words, matched unique words, and union count for both files.
11. Final Similarity Score – Displays the Jaccard Similarity percentage formatted to two decimal places.
12. Verdict – A text verdict based on the similarity threshold (HIGH PLAGIARISM DETECTED / Moderate / Low / Minimal).

Chapter 7: Important Features of the Project

7.1 Command-Line File Input

Users provide the paths to two plain-text (.txt) files as command-line arguments (`argv[1]` and `argv[2]`). The program validates argument count in `main()` and reports a clear usage message if they are missing. File existence is implicitly validated by `ifstream` — if a file cannot be opened, the Document object will have an empty word list.

7.2 Word Tokenization and Normalization

The `normalizeAndTokenize()` private method in the Document class processes the file content character by character. Each character is checked: if alphanumeric, it is appended to the current word after converting to lowercase using `tolower()`; if a delimiter (space, punctuation, newline), the current word is finalized. This ensures that words like 'Hello', 'hello,', and 'HELLO' are all treated as the identical token 'hello'.

7.3 Stopword Filtering

Before a word is stored in the words vector or `unique_words` set, it is checked against the `stop_words unordered_set` which contains the common English articles: a, an, the. Any word found in this set is silently discarded. This prevents extremely common words from inflating the similarity score between documents on similar topics.

7.4 Jaccard Similarity Score

The Jaccard Similarity Index is computed in `main()` using the unique word sets returned by `getUniqueWords()` from both Document objects. The intersection is found by iterating through `set1` and checking membership in `set2`. The union is calculated as $|\text{set1}| + |\text{set2}| - |\text{intersection}|$. The result is expressed as a percentage (0–100%) and printed with two decimal places.

7.5 Verdict Classification

Similarity Score	Verdict	Console Output
80% or above	High Plagiarism	HIGH PLAGIARISM DETECTED!
40% to 79%	Moderate Similarity	Moderate similarity found.
10% to 39%	Low Similarity	Low similarity.
Below 10%	Minimal / No Plagiarism	Minimal / No plagiarism detected.

7.6 Matched Word Display

The matched words vector is sorted alphabetically before display using `std::sort` from the C++ Standard Library. `TerminalRenderer::printMatches()` prints up to the first 20 matched words separated by commas, and if more than 20 exist, appends '... (and N more)' to keep the output concise.

7.7 Error Handling

The program handles common failure cases cleanly: insufficient command-line arguments (prints usage string and exits), both files empty or unreadable (prints error and exits), and division-by-zero protection in the Jaccard formula (`union_count` is clamped to a minimum of 1).

Chapter 8: Implementation

8.1 Step-by-Step Working

13. Program Launch: User runs plagiarism.exe with two .txt file paths as command-line arguments.
14. Argument Validation: main() checks argc < 3 and prints a usage error if arguments are missing.
15. Document Construction: Two Document objects (doc1, doc2) are created. Each constructor reads the file, normalizes content, filters stopwords, and populates the words vector and unique_words set.
16. Intersection Computation: main() iterates through doc1's unique_words set and checks each word against doc2's unique_words set using find(). Matches are stored in a matched_words vector.
17. Sorting: matched_words is sorted alphabetically using std::sort.
18. Union Calculation: union_count = set1.size() + set2.size() - matched_words.size().
19. Jaccard Calculation: similarity = (matched_words.size() / union_count) * 100.0.
20. Console Output: TerminalRenderer methods print matched words, statistics, similarity score, and verdict to the console in sequence.

8.2 Core Algorithm Pseudocode

```
Algorithm: main(argc, argv)
1. If argc < 3: print error and exit
2. doc1 = Document(argv[1])
   doc1 constructor:
       read file into content string
       normalizeAndTokenize():
           for each char in content:
               if alphanumeric: append tolower(c) to current_word
               else: if current_word not in stop_words:
                   words.push_back(word)
                   unique_words.insert(word)
3. doc2 = Document(argv[2]) [same as above]
4. for each word in doc1.unique_words:
       if word in doc2.unique_words:
           matched_words.push_back(word)
5. sort(matched_words)
6. union_count = |set1| + |set2| - |matched_words|
7. similarity = (|matched_words| / union_count) * 100.0
8. TerminalRenderer::printMatches(matched_words)
9. TerminalRenderer::printStats(doc1, doc2, ...)
10. TerminalRenderer::printResult(similarity)
```

Chapter 9: System Architecture

9.1 Console Application Architecture

Plagiarism Checker is a single-tier console application. Unlike web-based systems, there is no presentation tier (browser), application tier (web server), or network tier. All logic executes within a single compiled binary (plagiarism.exe) that runs in the user's terminal.

The architecture follows a clean two-class pipeline:

21. User provides two .txt file paths as command-line arguments.
22. Two Document objects independently read, tokenize, normalize, and index both files.
23. main() performs the set intersection and computes Jaccard Similarity.
24. TerminalRenderer prints all results to stdout.

9.2 Project Directory Structure

```
project/
+-- plagiarism.cpp      # Main C++ source (Document + TerminalRenderer +
main)
+-- compile.bat         # Windows compilation script (g++)
+-- plagiarism.exe      # Compiled binary (generated after compile.bat)
+-- sample/
    +-- doc1.txt        # Sample text file 1 for testing
    +-- doc2.txt        # Sample text file 2 for testing
```

9.3 Compilation & Execution Instructions

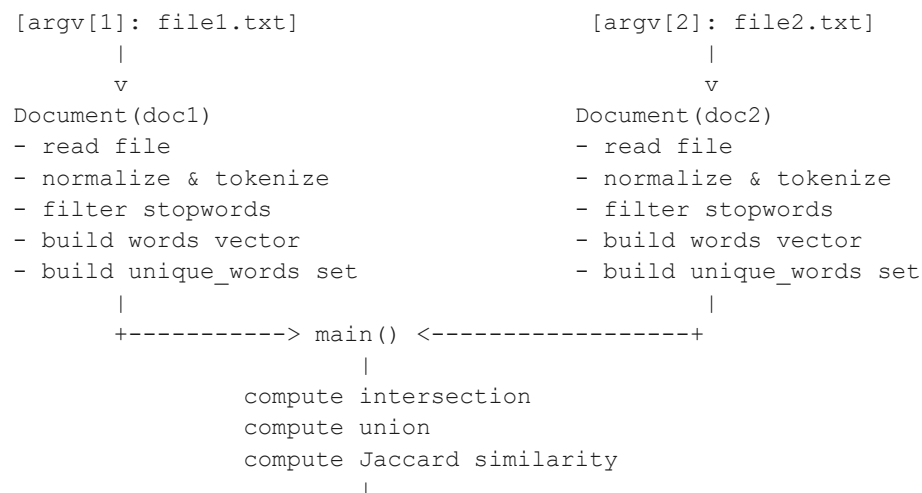
Windows (compile.bat):

```
# Step 1: Open Command Prompt in the project directory
compile.bat
# Step 2: Run the program
plagiarism.exe doc1.txt doc2.txt
```

Linux / macOS:

```
g++ -O2 -o plagiarism plagiarism.cpp
./plagiarism doc1.txt doc2.txt
```

9.4 Data Flow Diagram



```
      v
TerminalRenderer
- printHeader()
- printMatches()
- printStats()
- printResult()
      |
      v
Console Output (stdout)
```

Chapter 10: Coding and Libraries Used

10.1 C++ Standard Libraries Used

Library / Header	Purpose in Project
<code><iostream></code>	cout, cerr for console output and error messages
<code><fstream></code>	ifstream for reading .txt files from disk
<code><string></code>	std::string for file content and word storage
<code><vector></code>	std::vector<string> for ordered word list and matched words
<code><unordered_set></code>	std::unordered_set<string> for O(1) unique word lookup and stopwords
<code><algorithm></code>	std::sort for alphabetical sorting of matched words; std::min for display limit
<code><cctype></code>	isalnum(), tolower() for character-level word normalization
<code><iomanip></code>	setw() for column alignment; fixed and setprecision(2) for score formatting

10.2 Key Code Snippet – normalizeAndTokenize()

This private method in the Document class is the core tokenizer. It processes the raw file content character by character, builds words by accumulating alphanumeric characters converted to lowercase, and finalizes a word at every delimiter. Before storing, it checks the word against the stop_words set to filter common articles.

```
void normalizeAndTokenize() {
    string current_word = "";
    auto processWord = [&](string& word) {
        if (!word.empty()) {
            if (stop_words.find(word) == stop_words.end()) {
                words.push_back(word);
                unique_words.insert(word);
            }
            word = "";
        }
    };
    for (char c : content) {
        if (isalnum(c)) current_word += (char)tolower(c);
        else
            processWord(current_word);
    }
    processWord(current_word);
}
```

10.3 Key Code Snippet – Jaccard Similarity in main()

```
for (const string& word : set1) {
    if (set2.find(word) != set2.end())
        matched_words.push_back(word);
}
sort(matched_words.begin(), matched_words.end());
int union_count = set1.size() + set2.size() - matched_words.size();
if (union_count <= 0) union_count = 1;
double similarity = ((double)matched_words.size()
    / (double)union_count) * 100.0;
```

Chapter 11: Sample Console Output

The following screenshot shows the actual output produced by the compiled plagiarism.exe when run on two sample text files. The output demonstrates the full console report: the program header, common words list, analysis statistics table, the final Jaccard Similarity score, and the verdict.

```
=====
PlagCheck - Terminal Plagiarism Checker
=====

Common Words: and, are, arrays, data, efficiently, include, of, organizing, queues, stacks, storing, structures, types

--- Analysis Statistics ---
Total Words File 1:      26
Total Words File 2:      24
Unique Words File 1:     23
Unique Words File 2:     19
Matched Unique Words:    13
Union (Total Unique):     29
-----

>>> FINAL SIMILARITY SCORE: 44.83%
Verdict: Moderate similarity found.
```

Figure 1: Actual console output of Plagiarism Checker when comparing two sample .txt files.

In the screenshot above, the program compares two files and finds 13 matched unique words out of a union of 29. This produces a Jaccard Similarity score of 44.83%, resulting in a verdict of 'Moderate similarity found.' The common words list shows the 13 alphabetically sorted words present in both documents (and, are, arrays, data, efficiently, include, of, organizing, queues, stacks, storing, structures, types).

Note: The words 'and', 'of', etc. appear in the matched list because they were not in the built-in stopword set (which only filters a, an, the). This demonstrates a candidate area for future improvement through an expanded stopword list.

Chapter 12: Advantages and Limitations

12.1 Advantages

- **Pure C++ OOP Design:** The entire logic is organized into two well-defined classes (Document and TerminalRenderer), directly demonstrating encapsulation and abstraction.
- **Efficient Lookups:** Using `unordered_set<string>` for `unique_words` provides $O(1)$ average-case membership testing during intersection computation, making the comparison fast even for large files.
- **Zero External Dependencies:** No web server, Docker, Python, or browser is required. The project compiles and runs with a single `g++` command.
- **Jaccard Similarity Metric:** An industry-standard, mathematically sound similarity metric provides a reliable and interpretable percentage.
- **Stopword Filtering:** Common articles (a, an, the) are filtered out before comparison, reducing noise in the similarity score.
- **Cross-Platform:** Compiles and runs on Windows (MinGW), Linux, and macOS without code changes.
- **Alphabetically Sorted Matched Words:** The matched word list is sorted for easy reading and reproducibility.
- **Clean main():** Business logic is fully delegated to Document and TerminalRenderer, keeping `main()` simple and readable.

12.2 Limitations

- **Plain Text Only:** The system only accepts .txt files. PDF, DOCX, and other formats are not supported.
- **Word-Level Only:** Comparison is at the word level. Paraphrased content that uses synonyms or restructures sentences may not be detected.
- **Limited Stopword List:** The stopwords set contains only three words (a, an, the). Common words like 'is', 'are', 'of', 'and' are not filtered, which may inflate similarity scores for documents on similar topics.
- **No Internet Comparison:** The system compares only the two provided files; it does not check against online sources or databases.
- **Console Only:** Results are displayed as plain text in the terminal with no charts, graphs, or visual elements.
- **No Result Persistence:** Results are not saved; once the program exits, the comparison data is lost.
- **Single Comparison Session:** Only two files can be compared at a time; batch processing is not supported.

Chapter 13: Future Enhancements

25. Expanded Stopword Filtering: Extend the `stop_words` set to include all common English function words (is, are, was, were, of, and, or, in, on, at, to, for, etc.) to improve similarity accuracy for topically similar documents.
26. PDF and DOCX Support: Integrate document parsing libraries to support PDF and Microsoft Word formats, broadening the tool's applicability.
27. N-Gram and Phrase Matching: Extend comparison from single words to bi-grams and tri-grams to detect paraphrased content more effectively.
28. File Output / Report Export: Add an output flag (`--report result.txt`) to save the results to a text file for later review.
29. Color Console Output: Use ANSI escape codes to print colored output in the terminal (green for low similarity, red for high plagiarism).
30. Multiple File Comparison: Support batch comparison of multiple documents simultaneously, printing a similarity matrix.
31. Progress Bar: Add a text-based progress bar in the console output to visually represent the similarity score alongside the percentage.
32. GUI Version: Develop a graphical front-end using a C++ GUI library (Qt or SFML) for a more user-friendly interface.
33. Hash Map for Word Frequency: Track word frequency counts alongside unique word sets to weight common words more accurately in similarity scoring.
34. REST API / Web Version: Expose the plagiarism engine as a web service in a future extended version of the project.

Chapter 14: Conclusion

The Plagiarism Checker project represents a successful and meaningful application of Object-Oriented Programming concepts in C++ to a real-world problem, implemented as a console-based application that compiles and runs with a single command.

Through the development of this project, practical experience has been gained with OOP class design, encapsulation, abstraction, and the effective use of the C++ Standard Library. The `Document` class cleanly encapsulates all file I/O, tokenization, normalization, and set construction logic. The `TerminalRenderer` class separates all output formatting from the computation logic. The `main()` function acts as a clean orchestrator that connects the two classes and computes the Jaccard Similarity.

The project achieves all its stated objectives. The application accepts two plain-text files via command-line arguments, processes them through an OOP pipeline, computes Jaccard Similarity, and presents a well-formatted console report including similarity score, verdict, statistics, and sorted matched word list.

The use of `unordered_set<string>` for unique word storage demonstrates awareness of algorithmic efficiency — $O(1)$ average-case membership testing makes intersection computation fast even for large documents. The stopwords filtering feature further improves result accuracy by excluding noise words from the comparison.

The planned future enhancements — including expanded stopwords filtering, N-gram matching, color output, and batch processing — demonstrate the project's significant potential for growth into a more comprehensive plagiarism detection tool. The Plagiarism Checker is a tangible demonstration of how foundational OOP concepts translate directly into practical, compilable, and deployable software solutions.

Chapter 15: References

Online Resources:

35. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley. – Reference for C++ OOP, classes, standard library containers, and algorithms.
36. GNU Project. (2024). GCC, the GNU Compiler Collection. Retrieved from <https://gcc.gnu.org/>
37. Jaccard, P. (1912). The Distribution of the Flora in the Alpine Zone. New Phytologist, 11(2), 37–50. – Original paper defining the Jaccard Similarity Coefficient.
38. cppreference.com. (2024). C++ Standard Library Reference – unordered_set, vector, algorithm. Retrieved from <https://en.cppreference.com/>
39. GeeksforGeeks. (2024). Jaccard Index Similarity Measure – Data Mining. Retrieved from <https://www.geeksforgeeks.org/>
40. GeeksforGeeks. (2024). unordered_set in C++ STL. Retrieved from <https://www.geeksforgeeks.org/>
41. GeeksforGeeks. (2024). OOP in C++ – Classes and Objects. Retrieved from <https://www.geeksforgeeks.org/>
42. MinGW-w64 Project. (2024). MinGW for Windows. Retrieved from <https://www.mingw-w64.org/>

Other References:

43. Course Notes: OOPs Using C++ – BCA 2nd Semester, Dev Bhoomi Uttarakhand University.
44. Lectures and practical demonstrations by Ms. Dhruvi Dagar, Faculty, Department of Computer Applications.

— End of Report —

Plagiarism Checker | Gaurav Gunjan | BCA (AI&DS) 2nd Semester
Dev Bhoomi Uttarakhand University, Dehradun | April 2026